

Tugas Watermarking

Nama: Riko Satriya Giovanni

NIM: 18224108

Kelas: K02

Daftar Isi

[Daftar Isi](#)

[Detail Tugas Watermarking](#)

[Watermarking Implementation Plan](#)

[Garis Besar Proses Watermarking](#)

[Kode Pengerjaan](#)

[A. Bagian Import-import](#)

[B. Konfigurasi](#)

[C. Fungsi DCT](#)

[D. Membuat Watermark Biner](#)

[E. Proses Penyisipan Watermark](#)

[F. JPEG Compression Simulation](#)

[G. Proses Ekstraksi Watermark](#)

[H. Matrix Evaluasi](#)

[I. Visualisasi Hasil](#)

[J. Main](#)

Detail Tugas Watermarking

- Tambahkan watermark pada foto wajah sendiri.
- Watermark dapat berupa citra biner atau acak.
- Foto ini dikompres menggunakan kompresi JPEG.
- Evaluasi kinerja watermark dengan cara mengubah quality factor(QF) ke beberapa nilai. Tunjukkan nilai QF yang membuat watermark tidak dapat diekstrak.

Watermarking Implementation Plan

1. Objektif

Menyisipkan watermark (citra biner/acak) ke dalam foto wajah pribadi, memastikan ketahanannya terhadap kompresi JPEG, dan mengevaluasi batas kerusakannya berdasarkan variasi *Quality Factor* (QF).

2. Tech Stack

Bahasa: Python

Library: OpenCV/Pillow (I/O Image), NumPy (Matriks), SciPy (Algoritma DCT).

3. Alur Sistem (Arsitektur & Workflow)

A. Encoder (Penyisipan / Embedding):

1. Konversi foto RGB ke YCbCr (ambil channel Y/Luminance).
2. Bagi gambar ke blok matriks 8×8 .
3. Terapkan transformasi DCT.
4. Sisipkan bit watermark pada koefisien frekuensi menengah (menjaga balance kualitas dan ketahanan).
5. Terapkan Inverse-DCT dan simpan sebagai foto ter-watermark.

B. Simulasi Kompresi: Simpan foto ter-watermark dengan variasi parameter QF (misal: 90, 70, 50, 30, 10).

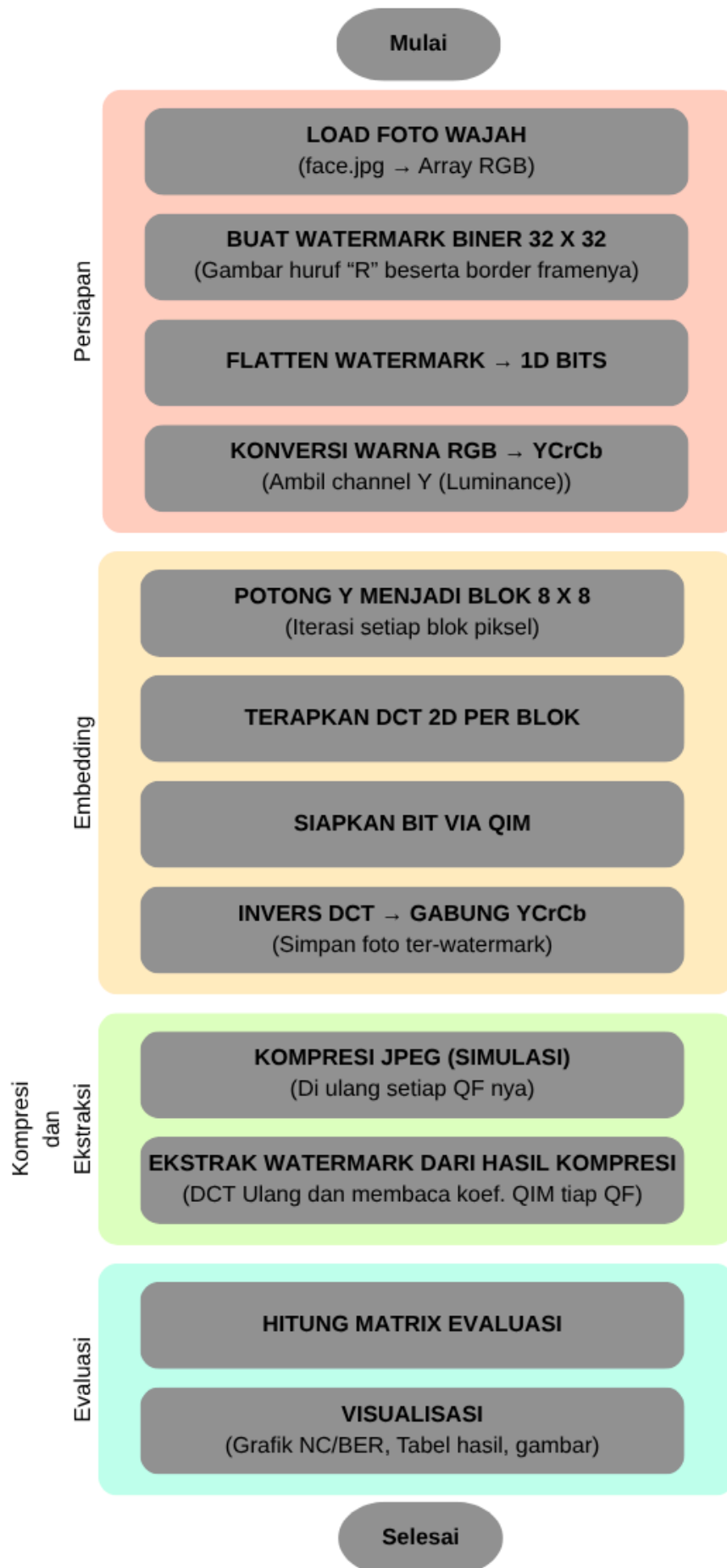
C. Decoder (Ekstraksi):

1. Terapkan ulang DCT pada foto hasil kompresi.
2. Ekstrak bit watermark dari lokasi koefisien frekuensi sebelumnya.
3. Susun kembali bit menjadi citra watermark utuh.
4. Metrik Evaluasi
 - PSNR: Menilai kemiripan visual foto asli vs foto ter-watermark (Target ideal $>30\text{dB}$).
 - NC (Normalized Correlation): Menilai akurasi watermark asli vs watermark hasil ekstraksi.

4. Target Output Tugas

- a. Visual: Bukti bahwa foto asli dan foto ter-watermark tampak identik oleh mata.
- b. Tabel Data: Tabel berisi Nilai QF, Visual Ekstraksi Watermark, dan Skor Akurasi (NC).
- c. Kesimpulan: Penentuan titik breakdown (misal: "Watermark tahan hingga QF 30, namun hancur/tidak terbaca pada QF 10").

Garis Besar Proses Watermarking



Dalam kasus kode saya, apabila memasukan foto face.jpg yang saya punya, Watermark hanya bisa di ekstrak hanya sampai QF =30, di QF ke-10, watermark sudah tidak dapat di ekstrak



Kode Pengerjaan

A. Bagian Import-import

```
import os
import io
import cv2
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from PIL import Image
from scipy.fftpack import dct, idct
```

▼ Penjelasan

1. Manipulasi Matriks & Matematika

- `import numpy as np`

Sebuah foto/citra pada dasarnya adalah matriks (kumpulan angka) yang merepresentasikan nilai piksel. *Library* ini melakukan pemrosesan citra untuk melakukan operasi matematika tingkat lanjut secara cepat. Library ini digunakan untuk mengubah gambar menjadi array, melakukan operasi penyisipan matriks watermark, hingga perhitungan eror evaluasi.

- `from scipy.fftpack import dct, idct`

Ini adalah fungsi utama untuk algoritma *watermarking* domain frekuensi.

- **dct** (*Discrete Cosine Transform*): mengubah blok piksel gambar (biasanya 8 x 8 piksel) dari domain spasial menjadi domain frekuensi. Di domain inilah kamu menyisipkan watermark.
- **idct** (*Inverse Discrete Cosine Transform*): mengembalikan blok frekuensi yang sudah disisipi watermark kembali menjadi gambar utuh yang bisa dilihat secara visual.

2. Pemrosesan Citra Digital

- `import cv2`

Ini adalah OpenCV (*Open Source Computer Vision*), *library* paling standar untuk manipulasi gambar. Library ini dipakai untuk membaca foto (`cv2.imread`), mengubah ruang warna (misalnya dari RGB ke Grayscale atau YCrCb), dan menyimpan gambar (`cv2.imwrite`) termasuk mengatur parameter kompresi QF.

- `from PIL import Image`

Pillow (PIL) adalah *library* gambar bawaan Python alternatif OpenCV. PIL digunakan berdampingan dengan OpenCV karena lebih mudah digunakan untuk manipulasi format JPEG (mengatur *quality factor*) menggunakan fungsi `save()`.

3. Visualisasi Hasil Evaluasi

- `import matplotlib.pyplot as plt`

Digunakan untuk membuat plot grafik atau menampilkan gambar di layar. Library ini saya pakai untuk menampilkan secara berjajar foto asli, foto yang sudah di-watermark, dan hasil ekstraksi watermark pada berbagai nilai QF.

- `import matplotlib.gridspec as gridspec`

Fitur dari Matplotlib yang membantumu mengatur tata letak grid (*layout*) gambar agar lebih rapi. Menampilkan satu gambar besar di atas, dan deretan gambar hasil ekstraksi watermark di bawahnya dengan rapi.

- `matplotlib.use("Agg")`

Perintah ini memberitahu Matplotlib untuk menggunakan *backend* "Agg" (Anti-Grain Geometry). Artinya, Matplotlib tidak akan mencoba membuka jendela *pop-up* (GUI) untuk menampilkan gambar, melainkan hanya merendernya di memori agar bisa disimpan langsung sebagai *file* gambar (misal: `plt.savefig`).

4. Operasi Sistem & Memori

- `import os`

Digunakan untuk berinteraksi dengan sistem operasi komputer. Misalnya, membuat folder baru untuk menyimpan hasil evaluasi, mengecek apakah file foto *selfie*-mu ada, atau membaca ukuran *file* (byte) sebelum dan sesudah kompresi.

- `import io`

Sangat berguna untuk proses *JPEG attack*. Alih-alih menyimpan gambar hasil kompresi berulang kali ke dalam *hardisk* (yang memakan waktu dan memenuhi ruang memori), kamu bisa menggunakan modul seperti `io.BytesIO()`. Ini memungkinkan kodemu mensimulasikan kompresi gambar dan membacanya kembali murni di dalam RAM (memori), sehingga proses iterasi penurunan QF (dari 100 ke 10) berjalan instan.

B. Konfigurasi

```
INPUT_IMAGE = "face.jpg"
OUTPUT_DIR  = "output"
BLOCK_SIZE  = 8
ALPHA       = 25
QF_LIST     = [90, 70, 50, 30, 10]

EMBED_COORDS = [(4, 1), (3, 2), (2, 3), (1, 4)]

os.makedirs(OUTPUT_DIR, exist_ok=True)
```

▼ Penjelasan

1. Parameter *Input* dan *Output*

- `INPUT_IMAGE = "face.jpg"`

Mendefinisikan nama *file* foto *selfie* yang akan disisipi watermark.

- `OUTPUT_DIR = "output"`

Nama folder tempat program akan menyimpan semua hasil kerjanya nanti (seperti gambar yang sudah di-watermark dan grafik hasil evaluasinya).

2. Parameter Utama Algoritma DCT

- `BLOCK_SIZE = 8`

Metode DCT bekerja dengan cara memecah gambar besar menjadi kotak-kotak kecil (*blocks*). Nilai 8 berarti gambar akan dipotong-potong menjadi blok berukuran 8×8 piksel.

- `ALPHA = 25`

Ini adalah **faktor kekuatan (Gain Factor)** dari watermark.

- Jika angkanya **dibesarkan** (misal: 50): Watermark akan tertanam sangat kuat dan tahan banting terhadap kompresi ekstrem, tapi foto wajah akan terlihat bertekstur atau rusak (*noise* terlihat).

- Jika angkanya **dikecilkan** (misal: 5): Foto wajah akan terlihat mulus seperti aslinya, tapi watermark sangat rapuh dan akan hilang meskipun hanya dikompres sedikit.

3. Parameter Pengujian (Attacking)

- `QF_LIST = [90, 70, 50, 30, 10]`

Ini adalah daftar nilai *Quality Factor* (QF) JPEG yang akan diuji. Program nanti akan otomatis melakukan *looping* untuk mengompres gambar dengan tingkat kualitas 90% (kompresi ringan) hingga 10% (kompresi sangat hancur) untuk melihat di titik mana watermark gagal diekstrak.

4. Lokasi Penyisipan Rahasia

- `EMBED_COORDS = [(4, 1), (3, 2), (2, 3), (1, 4)]`

Koordinat matriks di dalam satu blok 8×8 tempat menyimpan bit watermark. Koordinat ini sengaja dipilih di area **Mid-Frequency (Frekuensi Menengah)**.

- Mengapa tidak di *Low-Frequency* (ujung kiri atas)? Karena akan sangat merusak warna dan bentuk asli wajah.
- Mengapa tidak di *High-Frequency* (ujung kanan bawah)? Karena area itu adalah area pertama yang akan dibuang dan dihancurkan oleh kompresi JPEG. *Mid-frequency* adalah tempat persembunyian paling optimal.

5. Eksekusi Sistem

- `os.makedirs(OUTPUT_DIR, exist_ok=True)`

Perintah ini meminta sistem operasi (Windows/Mac/Linux) untuk membuat folder baru bernama "output" secara otomatis. Tambahan `exist_ok=True` mencegah program mengalami *error* atau berhenti secara paksa jika folder "output" tersebut ternyata sudah ada sebelumnya.

C. Fungsi DCT

```
def dct2(block):
    return dct(dct(block.T, norm="ortho").T, norm="ortho")

def idct2(block):
    return idct(idct(block.T, norm="ortho").T, norm="ortho")
```

▼ Penjelasan

1. `def dct2(block):` (Transformasi Maju)

Fungsi ini bertugas mengubah blok gambar (piksel berukuran 8×8) menjadi nilai frekuensi (koefisien DCT).

- **Cara kerja secara matematis:**

Untuk melakukan DCT 2D, matematika mensyaratkan melakukan transformasi dua kali. Kode tersebut pertama-tama melakukan *transpose* matriks (`block.T`) untuk memutar blok, lalu dikenakan fungsi `dct` pada barisnya, di-*transpose* kembali (`.T`), dan dikenakan fungsi `dct` lagi pada kolomnya. Hasilnya adalah matriks frekuensi 2D yang sempurna.

- **Arti `norm="ortho"` :**

Ini adalah normalisasi ortogonal. Sederhananya, parameter ini memastikan skala matematis tetap seimbang (energinya konstan). Tanpa parameter ini, saat kamu mengubah piksel menjadi frekuensi dan mengubahnya kembali, gambarnya bisa menjadi jauh lebih terang atau lebih gelap secara tidak wajar.

2. `def idct2(block):` (Transformasi Balik)

Fungsi ini adalah kebalikan persis dari `dct2`. IDCT singkatan dari *Inverse Discrete Cosine Transform*.

- **Fungsi dalam *watermarking*:**

Setelah kamu membongkar blok 8×8 menjadi frekuensi menggunakan `dct2` dan menyisipkan bit watermark rahasia di dalamnya, program tidak bisa menyimpan gambar tersebut ke dalam format `.jpg` dalam bentuk frekuensi. Gambar harus dikembalikan menjadi piksel yang bisa dilihat mata. Di sinilah fungsi `idct2` digunakan untuk "menyusun ulang" frekuensi yang sudah dimanipulasi tadi kembali menjadi blok piksel gambar utuh.

D. Membuat Watermark Biner

```
def generate_watermark(rows=None, cols=None) -> np.ndarray:
    """
    Watermark biner 32x32 berisi huruf 'R' yang presisi dan proporsional.
    Bit: 1 = putih, 0 = hitam.
    """
    WM_SIZE = 32
    wm = np.zeros((WM_SIZE, WM_SIZE), dtype=np.uint8)

    # — 1. Batang vertikal kiri (|) —————
    wm[6:26, 7:11] = 1

    # — 2. Garis horizontal atas dan tengah kepala —————
```



```

wm[6:10, 11:19] = 1
wm[14:18, 11:18] = 1

# — 3. Lengkungan kanan kepala (Bentuk 'D' dari huruf 'R') —
wm[7:17, 18:21] = 1
wm[8:16, 21:23] = 1
wm[9:15, 23:24] = 1
wm[10:14, 24:25] = 1

wm[10, 18] = 0
wm[13, 18] = 0

# — 4. Kaki diagonal kanan bawah (\) —————
for r in range(16, 26):
    c_start = int(13 + (r - 16) * 1.1)
    wm[r, c_start:c_start+4] = 1

# — 5. Border tipis (bingkai watermark) —————
wm[0, :] = 1; wm[-1, :] = 1
wm[:, 0] = 1; wm[:, -1] = 1

return wm

def watermark_to_bits(wm: np.ndarray) -> np.ndarray:
    return wm.flatten()

def bits_to_watermark(bits: np.ndarray, shape=(32, 32)) -> np.ndarray:
    n = shape[0] * shape[1]
    return bits[:n].reshape(shape)

```

▼ Penjelasan

1. `def generate_watermark(rows, cols):`

Fungsi ini bertugas membuat gambar watermark buatan sendiri (citra biner) secara otomatis menggunakan kode matematika, tanpa perlu memuat gambar dari luar.

- **Membuat Kanvas:** Baris `wm = np.zeros((WM_SIZE, WM_SIZE), dtype=np.uint8)` membuat kanvas hitam berukuran 32×32 piksel (semua bernilai 0). *Catatan kecil:* Parameter `rows` dan `cols` di dalam kurung fungsi sebenarnya tidak terpakai oleh kode di dalamnya, karena ukuran sudah dikunci secara statis di `WM_SIZE = 32`.
- **Menggambar Huruf 'R':** Kode di dalam *looping* `for` menggunakan perhitungan posisi (`norm_r` dan `norm_c`) untuk menggambar garis diagonal secara matematis..

- **Membuat Bingkai (Border):** Bagian bawah fungsi (`wm[0, :] = 1`, dst.) berfungsi menggambar garis putih di keempat sisi tepi kanvas agar watermark terlihat seperti memiliki bingkai kotak.

2. `def watermark_to_bits(wm):`

Fungsi ini digunakan dalam **Fase Penyisipan (Embedding)**.

- Gambar watermark berukuran 32×32 adalah data 2 Dimensi. Algoritma penyisipan kita membutuhkan antrean data bit tunggal yang panjang untuk dimasukkan satu per satu ke dalam blok foto *self*imu.
- Perintah `wm.flatten()` meratakan matriks 32×32 tersebut menjadi sebuah *array* 1 Dimensi (seperti pita panjang) yang berisi 1024 bit angka (deretan 0 dan 1).

3. `def bits_to_watermark(bits, shape=(32, 32)):`

Fungsi ini adalah kebalikan dari fungsi kedua, dan akan digunakan pada **Fase Ekstraksi (Extraction)**.

- Setelah berhasil membongkar dan menyedot deretan bit rahasia dari foto yang sudah dikompresi (yang bentuknya masih berupa pita panjang 1 Dimensi), kamu tidak akan bisa melihat bentuk aslinya.
- Perintah `bits[:n].reshape(shape)` mengambil 1024 bit pertama dari hasil ekstraksi, lalu menyusunnya dan melipatnya kembali menjadi bentuk matriks 2 Dimensi (32×32). Dengan begitu, hasil ekstraksi bisa ditampilkan ke layar sebagai sebuah gambar.

E. Proses Penyisipan Watermark

```
def embed_watermark(image_rgb: np.ndarray,
                    wm_bits: np.ndarray,
                    alpha: float = ALPHA) -> np.ndarray:
    """
    Sisipkan watermark ke komponen Y (Luminance) via DCT blok 8x8.
    Mengembalikan gambar RGB ter-watermark.
    """
    # Konversi ke YCbCr
    img_ycbcr = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2YCrCb).astype(
        np.float64)
    Y = img_ycbcr[:, :, 0]

    h, w = Y.shape
    bit_idx = 0
    total_bits = len(wm_bits)
```

```

for row in range(0, h - BLOCK_SIZE + 1, BLOCK_SIZE):
    for col in range(0, w - BLOCK_SIZE + 1, BLOCK_SIZE):
        if bit_idx >= total_bits:
            break
        block = Y[row:row+BLOCK_SIZE, col:col+BLOCK_SIZE]
        D = dct2(block)

        for (u, v) in EMBED_COORDS:
            if bit_idx >= total_bits:
                break
            bit = wm_bits[bit_idx]
            # Quantization Index Modulation (QIM) sederhana:
            # Koefisien dipaksa ke kelipatan alpha yang sesuai
            paritas bit

            coef = D[u, v]
            q = round(coef / alpha)
            if q % 2 != bit:
                q += 1 if (bit == 1) else -1
            D[u, v] = q * alpha
            bit_idx += 1

        Y[row:row+BLOCK_SIZE, col:col+BLOCK_SIZE] = idct2(D)

Y = np.clip(Y, 0, 255)
img_ycbcr[:, :, 0] = Y
watermarked_rgb = cv2.cvtColor(img_ycbcr.astype(np.uint8), cv
2.COLOR_YCrCb2RGB)
print(f" [Embed] {bit_idx} bit disisipkan dari total {total_b
its} bit watermark.")
return watermarked_rgb

```

▼ Penjelasan

Secara garis besar, fungsi `embed_watermark` ini bertugas untuk **menyembunyikan pesan rahasia (watermark)** yang berbentuk deretan bit (0 dan 1) ke dalam sebuah gambar.

Teknik yang digunakan yaitu menggabungkan **DCT (Discrete Cosine Transform)** dan **QIM (Quantization Index Modulation)**. Gambar dimodifikasi sedemikian rupa agar pesannya masuk, tapi perubahannya (idealnya) tidak disadari oleh mata manusia.

1. Konversi Warna (RGB ke YCbCr)

Pertama, gambar diubah dari format warna biasa (RGB) menjadi **YCrCb**.

- **Y** adalah *Luminance* (tingkat kecerahan/hitam-putih).
- **Cr dan Cb** adalah *Chrominance* (warna).

2. Memecah Gambar Menjadi Blok 8x8

Gambar tidak diproses sekaligus, melainkan dipotong-potong menjadi blok kecil berukuran 8×8 piksel (asumsi `BLOCK_SIZE` adalah 8). Cara ini mirip persis dengan cara algoritma kompresi JPEG bekerja.

3. Mengubah ke Domain Frekuensi (DCT)

4. Menyisipkan Bit Watermark dengan QIM (Inti Kodenya)

Di dalam blok frekuensi tersebut, kode memilih koordinat tertentu (`EMBED_COORDS`) untuk disisipi bit watermark. Metode yang dipakai adalah **QIM**, yang pada dasarnya adalah **permainan ganjil-genap**:

5. Mengembalikan Gambar ke Bentuk Semula

Setelah bit disisipkan, blok frekuensi (`D`) dikembalikan menjadi blok gambar biasa menggunakan `idct2()` (Inverse DCT).

Fungsi `np.clip(Y, 0, 255)` memastikan tidak ada nilai kecerahan piksel yang minus atau melebihi 255 (batas normal piksel). Terakhir, komponen Y digabung kembali dengan warnanya (CrCb) dan diubah menjadi gambar RGB seperti sedia kala.

F. JPEG Compression Simulation

Fungsi `jpeg_compress` ini bertugas untuk mensimulasikan efek kompresi gambar JPEG secara langsung di dalam memori komputer tanpa perlu menyimpannya menjadi file fisik di *hardisk*. Cara kerjanya dimulai dengan mengubah data gambar yang berbentuk matriks (*numpy array*) menjadi objek gambar Pillow, lalu menyimpannya secara virtual ke dalam ruang memori sementara (`io.BytesIO`) menggunakan format JPEG dan tingkat kualitas kompresi yang ditentukan melalui parameter `quality` . Setelah proses penyimpanan (yang secara otomatis memicu kompresi) selesai, kode akan "memutar

balik" kursor memori ke posisi awal (`buf.seek(0)`), membaca ulang gambar yang sudah terkompresi tersebut, dan mengubahnya kembali menjadi format matriks piksel awal untuk dikembalikan sebagai hasil akhir proses simulasi.

```
def jpeg_compress(image_rgb: np.ndarray, quality: int) -> np.ndarray:
    """Simulasi kompresi JPEG pada quality factor tertentu, kembalikan array RGB."""
    pil_img = Image.fromarray(image_rgb)
    buf = io.BytesIO()
    pil_img.save(buf, format="JPEG", quality=quality)
    buf.seek(0)
    compressed = np.array(Image.open(buf))
    return compressed
```

G. Proses Ekstraksi Watermark

```
def extract_watermark(image_rgb: np.ndarray,
                      total_bits: int,
                      alpha: float = ALPHA) -> np.ndarray:
    """Ekstrak bit watermark dari gambar (setelah kompresi)."""
    #Mengambil Komponen Y
    img_ycbcr = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2YCrCb).astype(np.float64)
    Y = img_ycbcr[:, :, 0]

    h, w = Y.shape
    extracted_bits = []
    bit_idx = 0

    #Membongkar Blok demi Blok (DCT)
    for row in range(0, h - BLOCK_SIZE + 1, BLOCK_SIZE):
        for col in range(0, w - BLOCK_SIZE + 1, BLOCK_SIZE):
            if bit_idx >= total_bits:
                break
            block = Y[row:row+BLOCK_SIZE, col:col+BLOCK_SIZE]
            D = dct2(block)

            #Membaca Pesan Rahasia
            for (u, v) in EMBED_COORDS:
                if bit_idx >= total_bits:
                    break
```

```

coef = D[u, v]
q = round(coef / alpha)
extracted_bits.append(int(q % 2))
bit_idx += 1

return np.array(extracted_bits[:total_bits], dtype=np.uint8)

```

▼ Penjelasan

Kode ini adalah kebalikan atau pasangan dari fungsi sebelumnya. Jika kode sebelumnya bertugas untuk **menyembunyikan** (*embed*) watermark, fungsi `extract_watermark` ini bertugas untuk **membaca atau membongkar kembali** (*extract*) deretan bit rahasia tersebut dari gambar, bahkan setelah gambar tersebut mungkin mengalami sedikit perubahan (seperti kompresi).

Cara kerjanya jauh lebih sederhana karena kita hanya perlu "membaca" tanpa perlu memodifikasi gambarnya. Berikut adalah penjelasan langkah demi langkahnya:

1. Menyiapkan Gambar (Mengambil Komponen Y)

Sama persis seperti saat menyisipkan, gambar RGB diubah dulu ke format **YCrCb**. Kita hanya mengambil lapisan **Y** (*Luminance* atau kecerahan) karena di sanalah watermark tersebut disembunyikan.

2. Membongkar Blok demi Blok (DCT)

Untuk menemukan pesan yang disembunyikan, kode harus mengikuti jejak yang sama persis dengan saat menyembunyikannya. Gambar dipotong-potong kembali menjadi blok 8×8 piksel, lalu setiap blok diubah kembali ke bentuk frekuensi menggunakan `dct2()` (Discrete Cosine Transform).

3. Membaca Pesan Rahasia (Inti Ekstraksi)

Di sinilah keajaiban metode **QIM (Quantization Index Modulation)** terlihat. Karena sebelumnya kita memaksa nilai frekuensi menjadi genap untuk bit `0` dan ganjil untuk bit `1`, sekarang kita tinggal mengeceknya:

- Kode melihat titik koordinat frekuensi tertentu (`EMBED_COORDS`).
- Nilai di titik tersebut (`coef`) dibagi lagi dengan nilai `alpha` yang sama, lalu dibulatkan menjadi `q`.
- **Pengecekan Ganjil-Genap (`q % 2`):** Jika `q` dibagi 2 sisa 0 (genap), maka bit yang diekstrak adalah `0`. Jika sisanya 1 (ganjil), maka bit yang diekstrak adalah `1`.
- Angka 0 atau 1 tersebut kemudian disimpan ke dalam daftar `extracted_bits`.

4. Mengembalikan Kumpulan Bit

Setelah kode berhasil mengumpulkan bit sebanyak `total_bits` (jumlah bit watermark yang kita tahu sejak awal), proses pencarian dihentikan. Kumpulan bit 0 dan 1 ini kemudian dikembalikan sebagai *array* (daftar) siap pakai, yang nantinya bisa diterjemahkan kembali menjadi teks, logo, atau bentuk watermark aslinya.

H. Matrix Evaluasi

```
def normalized_correlation(orig_bits: np.ndarray, ext_bits: np.ndarray) -> float:
    """NC: 1.0 = identik, 0.0 = tidak ada korelasi."""
    o = orig_bits.astype(np.float64) * 2 - 1 # {0,1} → {-1,+1}
    e = ext_bits.astype(np.float64) * 2 - 1
    denom = np.sqrt(np.sum(o**2) * np.sum(e**2))
    return float(np.dot(o, e) / denom) if denom != 0 else 0.0

def bit_error_rate(orig_bits: np.ndarray, ext_bits: np.ndarray) -> float:
    """BER: 0.0 = sempurna, 0.5 = acak total."""
    return float(np.mean(orig_bits != ext_bits))

def psnr(img1: np.ndarray, img2: np.ndarray) -> float:
    """Peak Signal-to-Noise Ratio antara dua gambar."""
    mse = np.mean((img1.astype(np.float64) - img2.astype(np.float64))**2)
    return 10.0 * np.log10(255**2 / mse) if mse != 0 else 10.0
```

▼ Penjelasan

1. Normalized Correlation (NC)

Fungsi ini mengukur tingkat kemiripan atau korelasi antara deretan bit watermark asli dengan bit yang berhasil diekstrak. Cara kerjanya adalah dengan mengubah nilai bit dari 0 dan 1 menjadi sinyal matematis -1 dan +1, lalu membandingkannya; di mana nilai akhir 1.0 berarti datanya identik sempurna, sedangkan 0.0 berarti tidak ada kemiripan sama sekali.

2. Bit Error Rate (BER)

Fungsi ini bertugas menghitung rasio tingkat kesalahan bit, yaitu persentase bit yang meleset atau berubah setelah diekstrak dibandingkan dengan aslinya. Nilai 0.0 menunjukkan proses ekstraksi yang sempurna tanpa cacat, sedangkan nilai 0.5 mengindikasikan bahwa data watermark sudah rusak parah dan bernilai acak total.

3. Peak Signal-to-Noise Ratio (PSNR)

Fungsi ini digunakan untuk mengukur seberapa jauh kualitas visual gambar menurun (atau seberapa banyak *noise* yang masuk) setelah disisipi watermark. Kodenya menghitung rata-rata kuadrat selisih piksel (MSE) antara gambar asli dan gambar yang ter-watermark; semakin tinggi nilai PSNR yang dihasilkan, semakin halus dan tak kasat mata perubahan yang terjadi pada gambar tersebut.

I. Visualisasi Hasil

Visualisasi yang dibentuk:

- **Visualisasi perbandingan visual:** Menampilkan kolase foto (asli vs ter-watermark vs terkompresi) sekaligus membandingkan gambar watermark asli dengan hasil ekstraksinya di setiap tingkat kompresi (QF) beserta status valid/rusaknya.
- **Visualisasi grafik metrik ketahanan:** Menampilkan dua kurva tren performa, yaitu grafik *Normalized Correlation* (NC) dan *Bit Error Rate* (BER) untuk melihat penurunan kualitas watermark seiring menurunnya kualitas gambar (QF).
- **Laporan rekapitulasi teks:** Menghasilkan tabel evaluasi berbentuk teks yang merangkum semua skor metrik (NC, BER, PSNR) beserta kalimat kesimpulan otomatis tentang batas maksimal ketahanan watermark.

```
def save_comparison_figure(original, watermarked, results, wm_orig_img, wm_extracted_list):  
    """Simpan figure besar: perbandingan gambar + watermark terekstrak per QF."""  
    n_qf = len(results)  
    fig = plt.figure(figsize=(5 * (n_qf + 2), 10), facecolor="#0f0f1a")  
  
    gs = gridspec.GridSpec(  
        2, n_qf + 2,  
        figure=fig,  
        hspace=0.4, wspace=0.3,  
        left=0.03, right=0.97, top=0.88, bottom=0.08  
    )  
  
    title_kw = dict(color="white", fontsize=11, fontweight="bold", pad=6)  
    label_kw = dict(color="#aaaacc", fontsize=9)  
  
    # — Baris 1: Foto Asli & Foto Watermarked —  
    ax_orig = fig.add_subplot(gs[0, 0])  
    ax_orig.imshow(original); ax_orig.axis("off")  
    ax_orig.set_title("Foto Asli", **title_kw)
```



```

ax_wm = fig.add_subplot(gs[0, 1])
ax_wm.imshow(watermarked); ax_wm.axis("off")
ax_wm.set_title(f"Foto + Watermark\nPSNR={psnr(original, water
marked):.1f} dB", **title_kw)

# — Baris 1: Foto terkompresi per QF —
for i, res in enumerate(results):
    ax = fig.add_subplot(gs[0, i + 2])
    ax.imshow(res["compressed"])
    ax.axis("off")
    ax.set_title(f"QF = {res['qf']}", **title_kw)
    ax.text(0.5, -0.04,
            f"NC={res['nc']:.3f} BER={res['ber']:.3f}\nPSNR=
{res['psnr_wm']:.1f}dB",
            ha="center", va="top", transform=ax.transAxes, **l
abel_kw)

# — Baris 2: Watermark asli & terekstrak per QF —
ax_wm_orig = fig.add_subplot(gs[1, 0:2])
ax_wm_orig.imshow(wm_orig_img, cmap="gray", vmin=0, vmax=1)
ax_wm_orig.axis("off")
ax_wm_orig.set_title("Watermark Asli (32×32)", **title_kw)

for i, (res, wm_ext) in enumerate(zip(results, wm_extracted_li
st)):
    ax = fig.add_subplot(gs[1, i + 2])
    ax.imshow(wm_ext, cmap="gray", vmin=0, vmax=1)
    ax.axis("off")
    status = "✓ VALID" if res["nc"] >= 0.5 else "✗ RUSAK"
    color = "#00ff99" if res["nc"] >= 0.5 else "#ff4444"
    ax.set_title(f"QF={res['qf']} {status}", color=color,
                fontsize=10, fontweight="bold", pad=6)

fig.suptitle(
    "Evaluasi Kinerja DCT Watermarking terhadap Kompresi JPE
G",
    color="white", fontsize=15, fontweight="bold", y=0.96
)
path = os.path.join(OUTPUT_DIR, "evaluation_result.png")
plt.savefig(path, dpi=150, bbox_inches="tight", facecolor=fig.
get_facecolor())
plt.close()

```

```

print(f" [Save] {path}")
return path

def save_metrics_chart(results):
    """Simpan grafik NC & BER vs QF."""
    qf_vals = [r["qf"] for r in results]
    nc_vals = [r["nc"] for r in results]
    ber_vals = [r["ber"] for r in results]

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5), facecolor="#0f0f1a")
    for ax in (ax1, ax2):
        ax.set_facecolor("#1a1a2e")
        ax.tick_params(colors="white"); ax.xaxis.label.set_color("white")
        ax.yaxis.label.set_color("white"); ax.title.set_color("white")
        for spine in ax.spines.values(): spine.set_edgecolor("#444446")

    # NC
    ax1.plot(qf_vals, nc_vals, "o-", color="#00ccff", lw=2.5, ms=8, label="NC")
    ax1.axhline(0.5, color="#ff6666", ls="--", lw=1.5, label="Threshold NC=0.5")
    ax1.fill_between(qf_vals, nc_vals, 0.5,
                    where=[v >= 0.5 for v in nc_vals], alpha=0.2,
                    color="#00ccff")
    ax1.set_xlabel("Quality Factor (QF)"); ax1.set_ylabel("NC")
    ax1.set_title("Normalized Correlation vs QF")
    ax1.set_ylim(-0.1, 1.1); ax1.legend(facecolor="#1a1a2e", labelcolor="white")
    ax1.set_xticks(qf_vals)

    # BER
    ax2.plot(qf_vals, ber_vals, "s-", color="#ff9944", lw=2.5, ms=8, label="BER")
    ax2.axhline(0.3, color="#ff6666", ls="--", lw=1.5, label="Threshold BER=0.3")
    ax2.fill_between(qf_vals, ber_vals, 0.3,
                    where=[v >= 0.3 for v in ber_vals], alpha=0.2,
                    color="#ff9944")

```

```

ax2.set_xlabel("Quality Factor (QF)"); ax2.set_ylabel("BER")
ax2.set_title("Bit Error Rate vs QF")
ax2.set_ylim(-0.05, 0.6); ax2.legend(facecolor="#1a1a2e", labelcolor="white")
ax2.set_xticks(qf_vals)

fig.suptitle("Grafik Metrik Ketahanan Watermark",
             color="white", fontsize=13, fontweight="bold")
plt.tight_layout()
path = os.path.join(OUTPUT_DIR, "metrics_chart.png")
plt.savefig(path, dpi=150, bbox_inches="tight", facecolor=fig.get_facecolor())
plt.close()
print(f" [Save] {path}")
return path

def print_report(results):
    """Cetak tabel evaluasi ke terminal dan simpan ke .txt."""
    header = f"\n{'='*62}\n  TABEL EVALUASI KINERJA WATERMARK (DCT-QIM,  $\alpha$ ={{ALPHA}})\n{'='*62}"
    row_fmt = "  QF={:<4} | NC={:.4f} | BER={:.4f} | PSNR_wm={:.2f} dB | {"}
    lines = [header]
    lines.append(f"  {'QF':<6}{ 'NC':>8}{ 'BER':>8}{ 'PSNR(dB)':>12} Status")
    lines.append("  " + "-"*56)

    valid_qf = []; broken_qf = []
    for r in results:
        status = "VALID ✓" if r["nc"] >= 0.5 else "RUSAK ✗"
        lines.append(row_fmt.format(r["qf"], r["nc"], r["ber"], r["psnr_wm"], status))
        (valid_qf if r["nc"] >= 0.5 else broken_qf).append(r["qf"])

    lines.append("  " + "-"*56)
    if valid_qf and broken_qf:
        lines.append(f"\n  Watermark TAHAN hingga QF = {min(valid_qf)}")
        lines.append(f"  Watermark RUSAK mulai QF = {max(broken_qf)}")
    elif not broken_qf:

```

```

        lines.append("\n    Watermark TAHAN di semua QF yang diuj
i.")
    else:
        lines.append("\n    Watermark RUSAK di semua QF yang diuj
i.")
    lines.append("="*62)

    report = "\n".join(lines)
    print(report)

    path = os.path.join(OUTPUT_DIR, "evaluation_report.txt")
    with open(path, "w") as f:
        f.write(report)
    print(f"    [Save] {path}")

```

J. Main

```

def main():
    # — Load gambar —————
    if not os.path.exists(INPUT_IMAGE):
        print(f"[INFO] '{INPUT_IMAGE}' tidak ditemukan — membuat f
oto dummy 256×256.")
        # Buat dummy wajah sederhana agar script bisa diteset tanpa
foto
        dummy = np.full((256, 256, 3), 180, dtype=np.uint8)
        cv2.ellipse(dummy, (128,110), (70,85), 0, 0, 360, (210,17
5,130), -1) # wajah
        cv2.circle(dummy, (105,105), 10, (80,50,30), -1) # mata
kiri
        cv2.circle(dummy, (151,105), 10, (80,50,30), -1) # mata
kanan
        cv2.ellipse(dummy, (128,155), (30,15), 0, 0, 180, (160,80,
80), 2) # mulut
        cv2.imwrite(INPUT_IMAGE, cv2.cvtColor(dummy, cv2.COLOR_RGB
2BGR))

    img_bgr = cv2.imread(INPUT_IMAGE)
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

    # Resize agar minimal cukup menampung watermark 32×32 bit
    # (butuh ≥ ceil(1024/4)=256 blok → 128×128 piksel minimum)
    min_dim = 256

```

```

h, w = img_rgb.shape[:2]
if h < min_dim or w < min_dim:
    scale = max(min_dim/h, min_dim/w)
    img_rgb = cv2.resize(img_rgb, (int(w*scale), int(h*scale)))

print(f"\n[Load] Gambar: {img_rgb.shape[1]}x{img_rgb.shape[0]} piksel")

# — Watermark —
wm_img = generate_watermark(*img_rgb.shape[:2])
wm_bits = watermark_to_bits(wm_img)
print(f"[WM] Watermark: {wm_img.shape}, {len(wm_bits)} bit")

# — Embedding —
print("\n[Step 1] Embedding watermark ke foto...")
wm_image = embed_watermark(img_rgb, wm_bits, alpha=ALPHA)

# Simpan foto asli & foto watermarked
cv2.imwrite(os.path.join(OUTPUT_DIR, "original.jpg"),
            cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR), [cv2.IMWRITE_JPEG_QUALITY, 95])
cv2.imwrite(os.path.join(OUTPUT_DIR, "watermarked.png"),
            cv2.cvtColor(wm_image, cv2.COLOR_RGB2BGR))

psnr_orig_wm = psnr(img_rgb, wm_image)
print(f" PSNR (Asli vs Watermarked): {psnr_orig_wm:.2f} dB")

# — Loop QF —
print("\n[Step 2] Kompresi JPEG & Ekstraksi Watermark...")
results = []; wm_extracted_list = []

for qf in QF_LIST:
    print(f"\n — QF = {qf} —")
    compressed = jpeg_compress(wm_image, quality=qf)
    ext_bits = extract_watermark(compressed, len(wm_bits), alpha=ALPHA)

    nc = normalized_correlation(wm_bits, ext_bits)
    ber = bit_error_rate(wm_bits, ext_bits)
    p = psnr(wm_image, compressed)
    print(f" NC={nc:.4f} BER={ber:.4f} PSNR_compressed={p:.2f}dB")

```

```

ext_img = bits_to_watermark(ext_bits, shape=wm_img.shape)
wm_extracted_list.append(ext_img)

# Simpan gambar terkompresi
out_path = os.path.join(OUTPUT_DIR, f"compressed_qf{qf}.jpg")

cv2.imwrite(out_path, cv2.cvtColor(compressed, cv2.COLOR_RGB2BGR),
            [cv2.IMWRITE_JPEG_QUALITY, 95])

results.append({"qf": qf, "nc": nc, "ber": ber,
               "psnr_wm": p, "compressed": compressed})

# — Visualisasi —————
print("\n[Step 3] Menyimpan visualisasi...")
save_comparison_figure(img_rgb, wm_image, results, wm_img, wm_
extracted_list)
save_metrics_chart(results)
print_report(results)

print(f"\n✅ Selesai! Semua output tersimpan di folder: ./{OUTPUT_DIR}/")

if __name__ == "__main__":
    main()

```